

LawMitra AI

AI-powered Indian Legal Guidance and Case-Routing Platform

Project Documentation Report

Project type: Front-end web application / legal-guidance prototype

Technology: HTML, CSS, JavaScript (ES modules), browser storage

Prepared for: Academic project submission and viva explanation

Project goal: Help citizens understand common legal and civic issues, preserve useful evidence, find the right escalation route, and draft a first complaint outline.

Important: LawMitra AI provides general information. It does not replace a qualified lawyer, legal aid office, or emergency service.

1. Project in Simple Words

LawMitra AI is a browser-based legal-help chatbot designed mainly for common Indian legal and civic problems. A visitor writes their problem in simple language. The system identifies the closest legal area, then shows practical next steps, evidence to save, possible remedies, nearby-help options, official links, and complaint-drafting support.

The project is intentionally a **guidance and case-routing system**, not a system that claims to decide a court case or give final legal advice.

Problem solved

- Many people do not know whether their problem is related to consumer law, cybercrime, property, employment, family law, or another area.
- They may not know what proof to preserve or where to complain first.
- They may need help in turning an incident into a clear written complaint.

Main objectives

1. Classify common legal and civic problems safely.
2. Give easy-to-read, structured first guidance.
3. Show evidence checklists, escalation options, and official verification sources.
4. Protect privacy by making saved chat history opt-in only.
5. Escalate urgent safety situations before normal case matching.

2. Technology Used

Part	Technology	Why it is used
User interface	HTML5 and CSS3	Creates the responsive landing page, chat screen, sidebar, dialogs, forms, and case catalogue.
Application logic	Vanilla JavaScript with ES modules	Keeps the project lightweight; no framework or package installation is required.
Legal routing	Keyword scoring and confidence checks	Routes a message only when the system has enough evidence; otherwise it asks for clarification.
Temporary memory	Session storage	Keeps short chat context and the selected State/UT in the active browser session.
Optional saved history	Cookies and local storage	Stores history only after an explicit YES choice.

Testing	Node.js assertion tests	Checks consent, intent routing, safety behaviour, privacy, and custom-catalogue functions.
---------	-------------------------	--

Why no backend? The current project is a static prototype. This makes it simple to run and reduces server-side data handling. A production version would need a secure backend, database, authentication, audit logs, and reviewed content publishing.

3. How the System Works

1. The user enters a problem in the chat box.
2. The input is cleaned: converted to lowercase, extra spaces removed, and repeated letters normalised.
3. The system first detects greetings, thanks, and goodbye messages.
4. For legal issues, it scores keywords against legal categories.
5. If confidence is high, it selects the closest case pattern.
6. If confidence is low or two categories are too close, it asks the user for more detail instead of guessing.
7. The response is displayed with remedies, evidence, next steps, contacts, map links, official sources, and drafts.

Simple architecture

Module	Responsibility
<code>app.js</code>	Connects the page UI, chat, saved history, map actions, State/UT selection, and local admin actions.
<code>ai/engine.js</code>	Main chat pipeline: classification, safety check, case match, and response generation.
<code>ai/intents.js</code>	Input normalisation, greetings, category keywords, confidence score, and safe fallback.
<code>ai/cases.js</code>	Curated legal case patterns, including summary, evidence, actions, escalation, and questions.
<code>ai/responses.js</code>	Turns a matched case into readable response sections and drafting templates.
<code>ai/help.js</code>	Helplines, map-search options, and official verification-source links.
<code>ai/history.js</code>	Opt-in history consent, browser UUID, storage, loading, and deletion.
<code>ai/jurisdiction.js</code>	Stores the optional State/UT selection for local-procedure reminders.
<code>ai/coverage.js</code>	Records privacy-safe signals for queries that the system cannot classify.

4. Legal Coverage

The current curated library has **43 case patterns across 38 categories**. It is broad coverage for common questions, not a promise to cover every Indian law or every state-specific procedure.

- Accident and compensation
- Banking and loans

- Business compliance
- Cheque and debt disputes
- Child welfare and adoption
- Civic nuisance
- Consumer disputes
- Contract disputes
- Criminal procedure
- Cyber fraud
- Defamation
- Disability rights
- Domestic and family matters
- Election and voter issues
- Environment
- Government/service matters
- Housing and RERA
- Immigration and visas
- Insurance
- Intellectual property
- Landlord and tenant issues
- Medical disputes
- Passport matters
- Police complaints
- Privacy and data misuse
- Property and inheritance
- Public records / RTI
- Public services
- Rights and discrimination
- Senior citizens
- Student issues
- Tax
- Telecom and internet
- Traffic
- Utilities
- Workplace and salary issues
- Insolvency
- Arbitration and mediation

5. Important Features Added

A. Confidence-based routing

The chatbot does not force every message into a legal category. For example, a greeting such as “hiiii” gets a friendly response. An unclear issue gets a request for more details. This avoids incorrect or alarming legal answers.

B. Jurisdiction awareness

Users can choose their State/UT in the privacy panel. The response then reminds them that procedure, forms, local authorities, and deadlines can vary by State/UT. The choice remains in the browser session only.

C. Official source links

Relevant responses show official places where the user can verify information. Examples include India Code for central laws, NALSA for legal aid, the National Cyber Crime Reporting Portal, RBI, MCA, Passport Seva, Election Commission, IP India, and the Income Tax portal.

D. Emergency and safety escalation

Messages containing immediate danger, violence, assault, threats, stalking, blackmail, self-harm, or similar safety indicators receive emergency guidance first. The response clearly points to **112** rather than treating the matter like an ordinary legal question.

E. Complaint and evidence support

For a matched case, the app can generate:

- Evidence checklist
- Immediate actions
- Possible remedies and escalation route
- Nearby-help map searches
- Ready-to-send email draft
- Short message draft

F. Privacy-safe coverage improvement

When the system cannot confidently classify a query, it records only non-sensitive signals such as confidence and matched keywords. It does **not** store the raw user message for this feature. The local export can be reviewed to decide which new categories should be added.

G. Development catalogue editor

The privacy panel includes a local JSON import/export tool for vetted case entries. It validates required fields such as ID, category, title, and summary. It is for development and review only; it is not a public admin panel.

6. Chat History and Privacy

User choice	System behaviour
Yes, save my history	Creates a persistent consent cookie and secure random browser ID. Conversations are stored locally in that browser and reload on future visits.
Not Now / No	No persistent NO preference or browser ID is stored. The dialog closes for the current page and appears again after refresh or a future visit.
Stop saving history	Removes the consent and browser-ID cookies. The user can enable history again later.
Delete all saved history	Deletes the saved conversations associated with that browser ID.

7. Testing and Quality Checks

The project contains automated Node.js tests. The following checks were run successfully:

Test file	What it checks
<code>tests/history.test.mjs</code>	New-user consent, YES persistence, UUID creation, saved history, delete history, NO behaviour, and prompt return after refresh.
<code>tests/intents.test.mjs</code>	Routing for newly added legal categories and safe fallback for an unknown issue.
<code>tests/platform-safety.test.mjs</code>	State/UT response note, official-source section, emergency 112 escalation, privacy-safe coverage gaps, and local-catalogue export.

Syntax checks were also performed on the main JavaScript modules using `node --check`.

8. How to Run the Project

1. Open a terminal in the project folder.
2. Start a simple local server: `python -m http.server 8080`
3. Open `http://localhost:8080` in a browser.
4. Try a common issue, for example: “My landlord is not returning my deposit” or “I lost money in a UPI scam.”

9. Limitations and Future Scope

- The app is not a lawyer and should not claim final legal advice.
- It cannot guarantee 100% legal coverage because laws, court decisions, and state procedures change.
- The current project has no backend, user authentication, shared database, or real legal-content approval workflow.
- A production version should add lawyer/editor review, a secure backend, role-based access, audit logs, versioned content publishing, rate limiting, and periodic source review.
- Future versions can support more Indian languages, document upload with strong privacy controls, state-specific forms, and connection to verified legal-aid workflows.

10. Viva / Professor Questions and Simple Answers

What is the main idea of LawMitra AI?

It is a legal-guidance chatbot that helps people identify common Indian legal issues and understand their practical next steps.

Does it use a large language model or external AI API?

No. This version uses a transparent rule-based keyword and confidence system with curated case data. This makes the routing easier to test and explain.

How does it avoid wrong categorisation?

It uses confidence scoring. If evidence is weak or categories are too close, it asks for clarification instead of guessing.

How is user privacy handled?

Chat history is opt-in. YES is saved persistently; NO is not stored persistently. Conversations remain in the user's own browser in this static version.

How are urgent messages handled?

Safety keywords are checked before normal case routing. The user is directed to 112 and immediate safety support.

What is the biggest limitation?

It is a guidance prototype, not a complete legal-advice system. It needs reviewed legal content and a secure backend before public production use.

11. Conclusion

LawMitra AI demonstrates how a simple, privacy-conscious web application can make legal information easier to access. It combines curated legal categories, safe confidence-based routing, evidence and complaint support, official links, emergency escalation, and user-controlled history. The project is suitable as an academic software project and can be developed further into a professionally reviewed legal-assistance platform.